

HerGuardian AI

System Architecture — Diagram, Pipeline, Data Flow & Design Decisions

System Architecture Diagram

The diagram below illustrates the complete system architecture of HerGuardian AI, spanning the client layer, the Next.js frontend, the Next.js API backend, the PostgreSQL data layer, and the external cloud services the platform depends on.

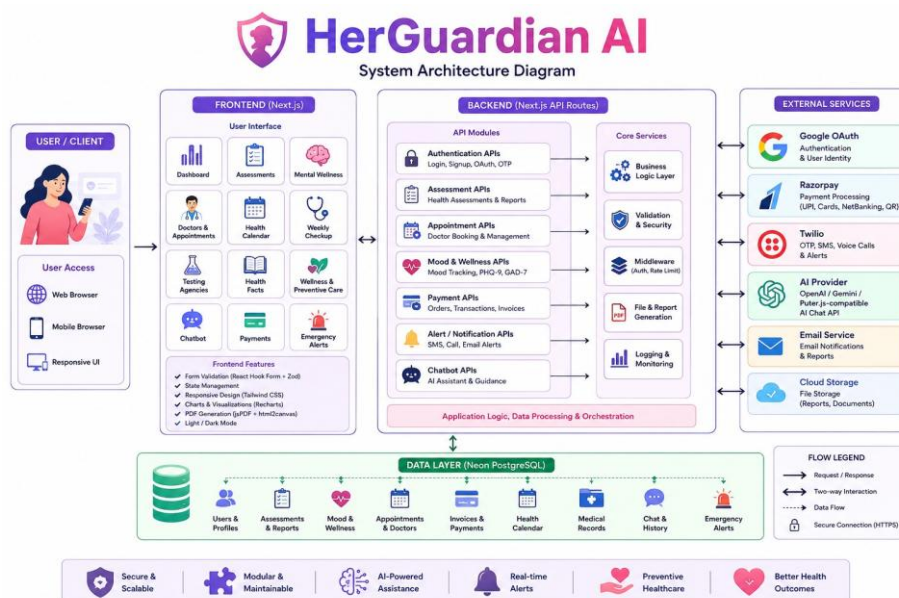


Figure 1: HerGuardian AI system architecture diagram — user/client layer, frontend, backend API modules, external services, and the PostgreSQL data layer.

Model / Processing Pipeline

Every user interaction — whether a form submission, a calendar update, a payment, or a chatbot query — follows the same general processing pipeline:

1. User Input via a frontend form, calendar, payment flow, or chatbot
2. Client-side validation and UX feedback (React Hook Form + Zod)
3. Routing to the relevant Next.js API route or local scoring logic
4. Processing against the database, an external API, or the AI provider
5. Generation of the result (score, appointment, report, response, or transaction)
6. Update of the dashboard, report, reminder, payment status, alert, or recommendation shown to the user



Figure 2: Model pipeline — from user input through frontend, validation, API/scoring logic, external services, generated results, to dashboard update.

Data Flow

7. User logs in or registers via Google OAuth or email/password.
8. User completes assessments, books appointments, logs symptoms, asks chatbot questions, or makes a payment.
9. The frontend validates input and sends it to the correct API route.
10. API routes process the data and persist records in PostgreSQL when DATABASE_URL is configured.
11. External services are invoked only when required — Google OAuth for login, Twilio for OTP/SMS/alerts, Razorpay for payments, the AI provider for chatbot responses, and Neon PostgreSQL for persistent storage.
12. The user receives results, recommendations, reports, calendar updates, reminders, or payment status.

Local vs. Cloud Components

Local Components (Client / Browser)

- Next.js frontend pages and React UI components
- Form validation
- Static educational content
- Local scoring and display logic
- Browser-based PDF generation
- Light/dark theme controls

Cloud Components

- Vercel deployment and hosting
- Neon PostgreSQL database
- Google OAuth credentials
- Razorpay merchant account
- Twilio messaging and calling account
- AI API provider for chatbot responses

Key Design Decisions

- Next.js App Router is used for both frontend routing and API routes, keeping the codebase unified.
- PostgreSQL was chosen because health, user, appointment, report, and payment data are inherently structured and relational.
- Drizzle ORM provides typed database schema management, reducing runtime errors.
- Razorpay was selected for its support of Indian payment methods — UPI, cards, QR, and net banking.
- Twilio handles OTP, SMS, and emergency call alerts.
- The chatbot is scoped for educational health support rather than medical diagnosis.
- The UI follows a healthcare-friendly, responsive layout with light/dark mode and accessible controls.
- The platform actively encourages professional consultation whenever moderate or serious symptoms are reported.